

Factory Cycle-Time Prediction With a Data-Mining Approach

Phillip Backus, Mani Janakiram, Shahin Mowzoon, George C. Runger, and Amit Bhargava

Abstract—An estimate of cycle time for a product in a factory is critical to semiconductor manufacturers (and in other industries) to assess customer due dates, schedule resources and actions for anticipated job completions, and to monitor the operation. Historical data can be used to learn a predictive model for cycle time based on measured and calculated process metrics (such as work-in-progress at specific operations, lot priority, product type, and so forth). Such a method is relatively easy to develop and maintain. Modern data mining algorithms are used to develop nonlinear predictors applicable to the majority of process lots, and three methods are compared here. They are compared with respect to performance in actual manufacturing data (to predict times for both final and intermediate steps) and for the feasibility to maintain and rebuild the model.

Index Terms—Due date, scheduling, statistical models, work-in-progress (WIP).

I. INTRODUCTION

SHORT-TERM cycle-time prediction is a well-documented problem in complex-process manufacturing such as semiconductor manufacturing. Such processes require hundreds of operations and often involve the integration of multiple products in a single manufacturing line. Furthermore, process strategies seek to optimize tool utilization causing a single tool to perform multiple operations. Product flow into these operations becomes complex as multiple product types at different stages of production compete for the same resource. In factory planning, managers use different scheduling strategies to generate rules for moving product through these complex processes. In many cases, real-time knowledge of the time it will take an individual lot to complete a subset of processes is of tactical importance.

In a completely linear process, intermediate cycle-time predictions can be obtained directly from Little's law [1]. However, such a static model does not typically predict well because it does not include the common stochastic behavior. For more complex processes, the most common solution is simulation. Due to the complexity of the simulations needed, an enormous amount of computing resources needs to be dedicated to the simulation in order to predict (in near real time) the cycle time for each lot in production for any given subset of operations. A complex simulation model can require substantial resources to be

adequately maintained to reflect process changes and it can be difficult to use to explore what-if scenarios.

Chung and Huang [2] characterized four methods to predict cycle time, and they provided several literature references for each type. Simulation has been the most commonly studied approach and a direct approach for semiconductor manufacturing was described by Wood [3] and Kim *et al.* [4]. Atherton and Atherton [5] argued that it is the best approach for a complex process such as semiconductor manufacturing. But, semiconductor manufacturing requires a complex simulation to be developed and maintained and this is a practical disadvantage. Furthermore, results tend to emphasize the average performance of the factory rather than the prediction for individual lots, which is the focus of our paper.

Statistical analysis is another approach, and a regression model was used by Raddon and Grigsby [6]. Traditional regression analysis does not handle categorical predictors, interactions, missing data, or outliers well. In addition, it is limited by inherently linear models. Our work also is based on a statistical analysis. However, we develop sophisticated predictor variables and we apply modern modeling methods. As was pointed out by Enns [7], statistical models need to be updated with the current tooling/capacity characteristics of the process. Consequently, we consider the maintenance of a model (in particular, the ability to regenerate the model) to be an important element. Advantages of a data mining approach were discussed by Janakiram *et al.* [8]. With the ability to quickly reanalyze the statistical data, such models can be updated as necessary.

Analytical methods rooted in queueing theory comprise the third class of methods, and this approach was applied by Chung and Huang [2]. Much process modeling is necessary to develop an effective model. Meng and Heragu [9] concluded that a 20% error in cycle-time prediction would be good for such an approach, even for a simple process, much less complex than semiconductor manufacturing. Consequently, this is not yet a practical solution. The last class of methods is really a hybrid or mixture of the previous three. For example, simulation and statistical analysis together was used by Kaplan and Unal [10]. Liao and Wang [11] used neural networks in combination with analytical methods for delivery time estimates. However, the process flow they considered was much less complex than our factory model analyzed here. Furthermore, considerable effort was needed to develop the analytical components of their delivery time model. Their analysis was based on simulated data. Also, the literature they cited did not attempt to model the actual data and process complexity of a real factory, as we do here.

Instead of an overly simplistic analytical model, a linear statistical model, or a complex simulation model, we propose a

Manuscript received March 16, 2005; revised February 6, 2006.

P. Backus is with Nissan North America, Gardenia, CA 90248 USA.

M. Janakiram and S. Mowzoon are with Intel Corporation, Chandler, AZ 85226 USA (e-mail: mani.janakiram@intel.com; shahin.m.movafagh@intel.com).

G. C. Runger is with the Department of Industrial Engineering, Arizona State University, Tempe, AZ 85287 USA (e-mail: runger@asu.edu).

A. Bhargava is with Honeywell, Phoenix, AZ 85034 USA.

Digital Object Identifier 10.1109/TSM.2006.873400

middle approach that uses historical data to provide intermediate cycle-time predictions. We seek to predict individual lot cycle time by comparing key characteristic of a lot in progress to lots that have completed the target operation for which predictions are to be made. Given that the production process is approximately constant over the time frame of prediction (say, a few weeks), then lots with similar characteristics should have similar cycle times. In this paper, we identify those characteristics and evaluate the quality of predictions.

In Section II, we describe the data structure used for this analysis. In Section III, we provide the features that were used in our models. Section IV summarizes the statistical methods considered for prediction, with an emphasis on decision trees. Section V provides performance results from actual process data.

II. PROCESS DATA STRUCTURE

Companies typically collect and store transactional data for each lot. A manufacturing execution system might be linked to other databases, and a data warehouse might be needed for an intermediate integration of the necessary information. Subsequently, one or more data tables can be created that form the raw transactional information for analysis.

A typical table contains rows for each operation and each lot in production. Each row has information to identify the lot, the current operation, the production route, the time in to the current operation, the time out from the previous operation, and the time out from the current operation. There are also variables describing the type of production lot, the product type, and the lot's priority. There can be hundreds of rows of data for *each* lot with more than 15 columns of data, giving more than 6000 variables to describe how an individual lot moves through the factory. Because operation move-in and move-out are time stamped, one of the first processing steps is to convert the time stamps into meaningful measures.

Because Little's law uses work-in-progress (WIP), cycle time (CT), and throughput (TP), it is important to create measures of those variables from the time stamps in the transactional database. We refer to the lot to be studied as the target lot (and this might also be called a tagged lot). A lot's cycle time for an operation can be described as queue time plus the tool processing time. Queue time is calculated as operation start minus previous operation out. Processing time is calculated as current operation out minus operation start. The following equations express these calculations in terms of the time stamps in the database. We label queue time (QT), process time (PT), operation in (OI), operation out (O), and previous operation out (PO):

$$CT_{\text{operation}} = QT + PT$$

$$QT = OI - PO$$

$$PT = O - OI.$$

The time of entry of the target lot into a tool depends on the queue and also upon lots that will arrive at the queue prior to the target lot's entrance into the tool. The latter are important

because they might be higher priority in the scheduling/dispatching used by manufacturing. The added complexity of adaptive scheduling priority causes WIP to be a dynamic measurement. Consequently, several different estimates of WIP are potentially useful to cycle-time prediction. One potentially useful measure is the queue length at the time of the target lot's arrival. Other estimates of WIP should consider the priority of the target lot. If one has information for lot priority, then lots with lower priority in the queue can essentially be ignored, except for lots currently being processed. We extracted information for lot priority from the progress of the target lot through the queue, but this was a difficulty due to the nature of our data, and this extra complexity is not presented here.

We performed additional processing to reduce the number of variables associated with each lot. Instead of considering each operation in the process, we summarized the data over the critical operations or bottlenecks in the process. Process engineers familiar with the production process validated operations which were considered to be the most critical and at which bottlenecks in the process flow formed. Queue time and process time were summed across all intermediate operations, giving an aggregate cycle time between the critical operations (that is also referred to as *lumping* operations). Let the subscript i indicate the i th intermediate step between consecutive bottlenecks, and let b indicate the number of process operations corresponding to a bottleneck, then

$$\text{Aggregate CT} = \sum_{i=1}^b CT_i.$$

We created a vector of aggregate cycle times for each critical step in the manufacturing process. Throughout this paper, we will refer to this vector of aggregate cycle times as a lot velocity vector. For purposes of notation, we call the individual elements of the vector $V_{\#}$ where $\#$ refers to the ordered bottleneck. For example, V_{12} refers to the aggregate cycle time from the end of the 11th critical step to the end of the 12th critical step. Also, the measures of WIP were calculated as described previously but relative to move-in and move-out of the critical operations.

Finally, the rows of data describing the individual steps for each lot were combined into a single row. The columns were then lot id, lot start date, lot end date, lot production type, product id, lot priority, production route, measures of WIP, CT, and the time stamps for each of the 20 different critical steps. Of the 160 columns, only 118 were used as possible predictor variables. The other 42 columns were related to the time stamps.

Two filters were applied to the data set. Only lots that had complete information from each operation from process start to finish and were identified as production lots were used. Lots that did not have complete information were eliminated because we were not able to compare our predictions to actuals at every step of the process. Test or engineering lots were eliminated because these lots were handled differently than normal production lots and could be arbitrarily placed on hold or advanced as determined by sources outside of the production process. However, all filtered lots were included in the WIP counts since their presence affects the cycle time of production lots.

III. FEATURES AND CHARACTERISTICS OF CYCLE-TIME PROBLEM

One could predict cycle time by comparing a new lot's cycle time at certain (critical) steps in the process to lots that had completed the process. This provides a pattern of times for the target lot. An average of cycle times for the most similar lots would predict the cycle time for the target lot. This method depends on the choice for the most similar patterns and further discussion is provided as follows.

Little's law states that, on average, $CT = WIP/TP$ [1]. Simulation methods model lot flow through the factory based on Little's law using probability distributions to capture the variation in the system (from sources such as processing times, probabilistic routing due to rework, setup times, and so forth). In the process studied in our analysis, the precision and reliability of the machines enable one to assume throughput to be nearly constant for a given machine. With this simplifying assumption, the cycle times of lots processed by the same tool are proportional to the ratio of the WIP. That is, the cycle time for lot b equals the ratio of the WIP multiplied by the cycle time for lot a . In a linear production process with a first-in/first-out (FIFO) scheduling rule, WIP would remain constant for each lot at each tool; consequently, the cycle time of a lot for the series of tools would be proportional to the ratio of WIP. The accuracy of this prediction would depend on the variability in machine throughput. However, many production processes allow for reentrant flows, in which case WIP for a lot is not constant at every tool. In some cases, a lot may leave a tool at one step and return to the same tool several steps later. The WIP for the second pass through the tool includes new lots that were introduced into the factory and are making their first pass through the tool. Hence, the naive approach of simply multiplying the summands of all tool cycle times by the WIP ratio fails to work since the WIP of the lot to be predicted is unknown. Although a simple proportional relationship between cycle times of lots is not expected, lots that encounter similar values of WIP should have similar cycle times.

Also, because the cumulated time increases with the successive steps of the process, it is more useful to use the difference in completion times between successive steps. This velocity vector is available for each lot and its dimension depends on the step of the lot at the time of prediction. For example, for a lot that completed step i , there are $i - 1$ differences in completion time available. To capture the most recent status, this vector can be truncated to the most recent, say m , velocities.

Another prediction strategy models the factory state. Given a lot at step j , the WIP at neighboring steps, say $j - m$ to $j + m$, can be used as a predictor. A target lot can be matched to completed lots that encountered similar WIP states when they were at step j , and the average cycle time of the most similar lots can be used for prediction. This method replaces the velocity vector with the vector of WIP states for the target lot. The neighborhood should be selected large enough to capture reentrant flows. We consider the following example. For target lot t at step 10 in the process, we calculate the WIP for t in queue for step 11 and count the lots behind and ahead of t , say for processes 8–10 and 12–14. Using a multivariate measure of similarity, we compare lot t to lots

that have already completed the entire manufacturing process and use their average as the predicted cycle time of lot t .

From the transactional data it is easier to derive the WIP state, or at least an estimate of it, than to explicitly collect the data. To that end, averages of cycle times for adjacent steps, within a window of time, provide information about throughput and WIP. For the steps just ahead of the target lot, the average cycle time actually provides information about both the WIP ahead of the target lot and a measure of the throughput and WIP for prior lots. If an appropriately sized window in time is constructed to calculate the average WIP at the next operation, then over the window the average cycle time is

$$\overline{CT} = \frac{\sum_{i=1}^n CT_i}{n} \quad (1)$$

where n is the number of lots in the window of time. Note that if the window is constructed to include lots that finish the current operation before the target lot and do not leave the operation before the target lot enters, then n counts all such lots and this equals the WIP for the target lot. Hence

$$\overline{CT} = \frac{\sum_{i=1}^n CT_i}{WIP}. \quad (2)$$

The role of this result in the prediction model is discussed in the following section.

Given a target lot at step j , we can calculate the average cycle time of neighboring lots at steps $j - m$ to $j + m$. We extend the notation to denote the cycle time for lot i through step j as $CT(i|j)$. Indicator functions are used to express the calculation

$$WIP \text{ Surrogate}_{j-m} = \frac{\sum_i CT(i|j-m)I(i)}{\sum_i I(i)}$$

where the indicator function $I(i)$ is defined as one if

$$\begin{aligned} \text{Time Out}(\text{lot } t \text{ step } j) - 24 \text{ hrs} &\leq \text{Time Out}_i \\ &\leq \text{Time Out}(\text{lot } t \text{ step } j) + 24 \text{ h} \end{aligned}$$

and zero otherwise. In the following section, we refer to elements of this vector as $C\#$, where the definition of $\#$ remains consistent with the notation for the elements of the velocity vector described in Section II.

Real-time intermediate cycle-time prediction presents several interesting problems. Challenges arise from the amount of available information and the quality of the predictors. If the cycle times between critical processes are used as a predictors and a prediction is made from the j th critical operation, then, as mentioned previously, the number of predictors is $j - 1$; this number changes with the progression of the lot through the process.

The length of the prediction also depends on the starting point of prediction. As a lot moves closer to the end of the process, the variability in the cycle-time prediction decreases. For a lot to start operation j with m total operations the total cycle time equals $CT_j + \dots + CT_m$. Each intermediate cycle time has an

associated variance. The variance of cycle time from the j th operation to process completion is

$$\text{Var} \left(\sum_{k=j}^m \text{CT}_k \right) = \sum_{k=j}^m \text{Var}(\text{CT}_k) + \sum_{k=j}^m \sum_{h=j}^m \text{Cov}(\text{CT}_k, \text{CT}_h) \quad k \neq h.$$

In the data considered here, the covariances between intermediate cycle times were positive. Hence, the variance of the sum of intermediate cycle times increases as the number of intermediate steps increase. As expected, predictions nearer to the end of the process have a lower error because the amount of variability has decreased. The increase in variability not only depends on the variances of the individual steps but also on the covariances, and the increase can be a complex function of the number of intermediate steps.

The first challenge is handled differently depending on the type of model used to fit the data. This is discussed as follows. The second challenge cannot be addressed by the model choice. However, when making comparisons of performance at different prediction points, the measure of performance must account for the natural decrease in variation as the prediction point moves nearer the final step.

IV. PREDICTION METHODS

Many statistical models exist for prediction. In general, lots with the most similar patterns from historical data are used to predict the target lot. Several different statistical methods were applied and results were compared. This section outlines the advantages and disadvantages of clustering, K-nearest neighbors, and regression trees. We consider both the properties of the technique and the facility of implementation in a real-time prediction system. A successful model should provide predictions that are accurate enough to be used in factory planning and lend itself readily to implementation. Neural networks had the disadvantage that they required extensive time to train. Furthermore, they did not adequately handle categorical predictors. Product type was an important predictor and neural networks require a somewhat artificial numerical score be assigned to product type in order to incorporate it into the model. Also, neural networks were further complicated by the comments in the previous section that more information was acquired as a lot moved through production. Tables I and II do not provide a comprehensive list of advantages and disadvantages, but rather they outline those characteristics most relevant to cycle-time prediction. Standard references for these methods include Hastie *et al.* [13] for a statistical view and Mitchell [14] for a computer science view. Also, the specific regression tree algorithm we used was CART described in the book by Breiman *et al.* [15].

Each of these three methods creates a partition of the training data, grouping observations together based on some criteria of similarity. With clustering and nearest neighbors, the measure of similarity is based on the predictor variables. For regression trees, the similarity is a measure of the distance of response variable from the mean of the response of other observations at a

TABLE I
COMPARISON OF MODEL CHARACTERISTICS

Method	Advantages	Disadvantages
Clustering: Form groups of lots with similar characteristics through the completed operations and then use the mean cycle time for each cluster as the predicted cycle time. A common measure of similarity is the Euclidean distance between the predictor values for target lot and the historical lots. Furthermore, different measures such as complete linkage, average linkage, and Ward's distance are commonly used to create clusters.	Matches physical concept of similar lots. Easily handles the change in the number of predictors as a lot moves through the process.	Difficult to use both categorical and continuous data. Clusters are formed without considering the final cycle time of lots.
K-Nearest Neighbors: Searches the historical data for lots with similar characteristics to the target lot. A common measure of similarity is the Euclidean distance between the predictor values for target lot and the historical lots. Average cycle time of the K most similar lots is used as the predicted value for the target lot. Several different values of K can be considered to tune the algorithm. Here, $K = 10$ was found to provide the best predictions.	Robust to outliers in predictor variable space. Generally have high predictive power for moderate number of predictors. Handles missing values well.	Difficult to use both categorical and continuous data. Typically do not perform well for high-dimensional data.
Regression Trees: Create predictions based on a recursive binary rule. The final node or leaf of the tree is the average of all data that is determined to belong to that node based on the series of rules created. Splitting rules are created by minimizing the squared loss function. Several different algorithms exist for creating a tree and in our analysis we used the CART algorithm with default criteria to select the size of the tree.	Handles both categorical and continuous data well. Handles missing values well. Robust to outliers in predictor variable space. Provides information about importance of predictor variables. Fast and simple.	High variance of tree structure. A robust loss objective can be easily incorporated to handle outliers in the measured response.

given node of the tree. We also considered a hybrid method that combines both clustering and regression trees. First, we used a clustering algorithm to create clusters of similar lots. Then, within each cluster we applied CART to construct a regression tree. The result is a method that measures the similarity of lots based on their associated vector of features and further divides the lots into groups that have similar total cycle times. This combination adds additional complexity but improves the predictions.

V. RESULTS

In general, we found that with the metrics we created the data could be partitioned in such a way as to provide good predictions

TABLE II
COMPARISON OF MODEL IMPLEMENTATION

Method	Advantages	Disadvantages
Clustering	If cluster assignment is based on Euclidean distance from cluster center, prediction of multiple steps can be made from any current step using only one set of clusters.	User interaction needed to determine optimal number of clusters. Clusters need to be updated as key characteristics of product or factory change.
K-Nearest Neighbors	Prediction of multiple steps can be made from any current step by using same nearest neighbors. No need to update model since most recent data is incorporated in available neighbors.	User interaction needed to determine optimal number of neighbors.
Regression Trees	Automated handling of optimal tree size. Fast and easy to recompute model.	Tree must be constructed for each step to be predicted from. Trees must be updated as key characteristics of factory and product change.

for future observations. The accuracy of the prediction varied from method to method.

A. Performance Measures

We evaluated performance based on the following measures: mean squared error, mean absolute error, median absolute error, and median percentage error. Since the data set contained over 1000 lots, we defined mean squared error as

$$\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n} \quad (3)$$

where y_i is the observed cycle time of the i th row, $\hat{y}_i$ is the predicted cycle time, and n is the number of observations. We did not divide by df , the degrees of freedom, and this has a small effect for the large sample size used here. Also, mean absolute error is

$$\frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

and median absolute error is

$$\text{Median}(|y_i - \hat{y}_i|).$$

B. Comparison of Performance

Table III demonstrates the performance of the different methods from a selected prediction point, approximately from the last one-third of the process. We note that there is a rather distinct difference between the median absolute error and the mean absolute error. We attribute this to a small number of lots whose behavior is quite different from the majority of the lots. We address this further in the following section. From Table III, we determine that CART produces the lowest median and mean absolute error and when combined with clustering it performs even better. The error reported is a proportion of a

TABLE III
METHOD PERFORMANCE FOR PREDICTION OF FINAL CYCLE TIME FROM INTERMEDIATE STEP. ERROR AS PROPORTION OF FIXED CYCLE TIME ON TEST DATA

Method	Median Absolute Error	Mean Absolute Error
KNN (K = 5)	0.070	0.16
KNN (K = 10)	0.078	0.16
CART	0.042	0.09
Cluster (N = 5)	0.098	0.12
Cluster (N = 10)	0.087	0.11
CART in Cluster	0.036	0.06
Neural Network	0.085	0.12

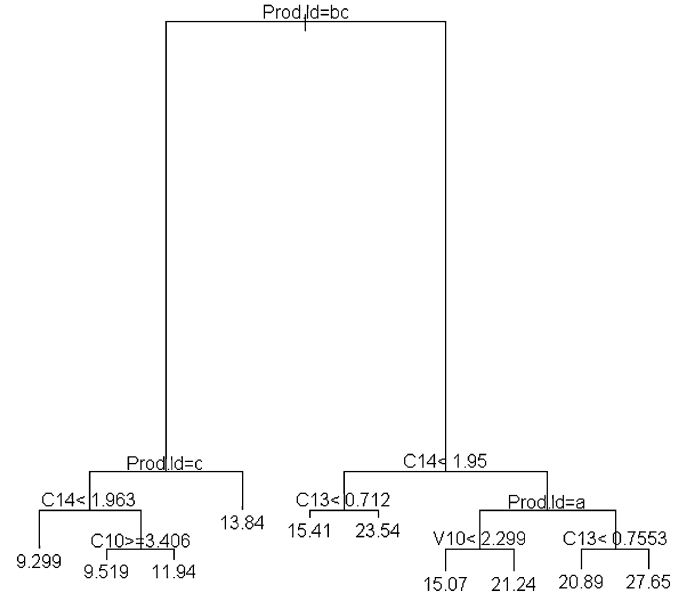


Fig. 1. Tree diagram for CART (outliers included).

fixed cycle time. The results should only be compared relative to each other and not to an absolute standard. Data was divided randomly with 50% for training and 50% for testing. The reported error was calculated from the testing data. CART used a ten-fold cross validation technique within the training data to select the final tree. Fig. 1 displays the diagram of the tree created by CART. At each node a rule is given. For numerical predictors, inequalities are used and the rule splits lots having a value greater than the constant to the right and lots with a value less than (or equal to) to the left. In the case of categorical predictors, if the lot meets the criterion, it splits to the left; otherwise, it splits to the right. The length of the branches between each node is proportional to the amount of variability explained by the splitting rule.

Predictor variables used in the analysis were coded as follows. Product ID is a categorical predictor with four levels (a, b, c, d) corresponding to four unique products. Also, $C\#$ is a continuous predictor which is the average cycle time (throughput) for the set of steps from critical step $\# - 1$ to critical step $\#$ for lots having completed that group of steps within a defined time window from when the target lot finished the current critical

step. Finally, $V_{\#}$ is a continuous predictor which is the actual cycle time for the target lot to complete all steps from the end of the $\# - 1$ critical step to the current $\#$ step.

The tree structure lends itself well to interpretation. The first split in Fig. 1 divides lots into two groups based on the type of product. Other important variables in the splits are the average cycle times for neighboring critical operations as well as the cycle time of the current operation for the target lot. The average cycle time of a prior critical process is also included in one node of the tree. We interpret the selection of these variables in the tree in the following manner. The cycle time of a lot depends on its own rate of progress as well as the progress of lots both in front and behind it.

The number associated with the terminal (leaf) node of the tree is the predicted cycle time for a lot that is assigned to that node based on values of the predictor variables. These predicted cycle times are means in the coded units used for this analysis. One can notice from Fig. 1 that lots from products *b* and *c* are assigned to the left-most leaves in the tree and predicted to have shorter cycle times than the other product types.

C. Outlier Detection and Robust Prediction

During the manufacturing process, a number of events occur which cause a percentage of the lots to behave very differently than an average lot. An engineer may place a lot on hold for revision, machines may need unscheduled repairs, or a lot may return to previous processes for rework. The data provided did not contain information about these events; consequently, we could not distinguish these lots from typical lots. We did not attempt to model the cycle time for such lots. Furthermore, these lots also impact the predictions for other lots. Each of the prediction methods averaged over a partition of the lots and such a mean is not robust to outliers. Consequently, predictions are worse when the unusual lots are included in calculating the mean. Our objective for this analysis of actual data was to predict cycle times well for the majority, but not all, the lots in the process. No attempt was made in this analysis to predict what particular lot might be put on hold for engineering reasons. Instead, we focused on good predictions for the bulk of normal lots and through this objective provide value to manufacturing.

Also, we considered several different technical corrections to remedy the effect of extreme lots. First, we considered screening the data prior to analysis to identify extreme lots. A common technique is to consider the cycle time of all lots and identify those whose cycle time is greater than three standard deviations from the mean of all cycle times. The implementation of this rule was quite simple, and it improved predictions for all three methods. A disadvantage of such a technique is that we may lose the opportunity to discover a rule for separating these lots (and important information about why the cycle times are so extreme). Furthermore, neither a global mean nor global standard deviation are necessarily good metrics to screen for local outliers in a particular partition of lots.

Another technique replaces the cluster, neighborhood, or tree node mean with the median to provide a more a robust summary. Implementation is more difficult since most of the standard software packages do not allow a change from mean to median. For decision trees, this also changes the objective function

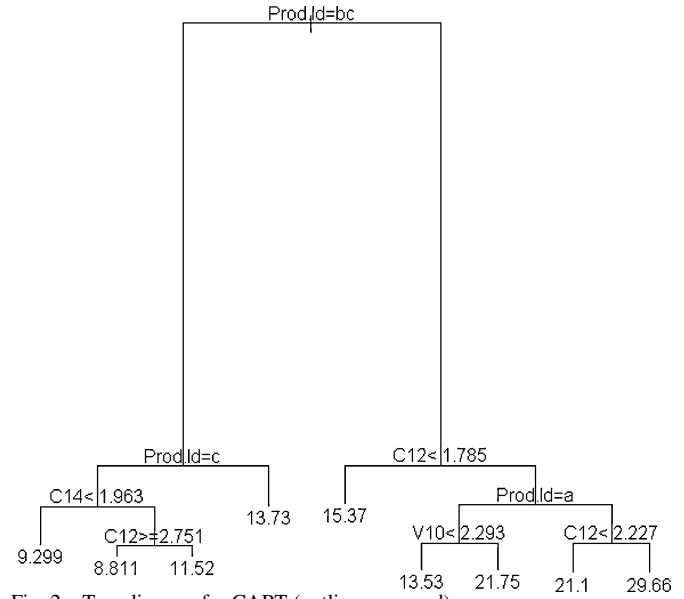


Fig. 2. Tree diagram for CART (outliers removed).

TABLE IV
COMPARISON OF PREDICTIONS REMOVING OUTLIERS AND USING NODE
MEDIAN VERSUS NODE MEAN. ERROR AS PROPORTION OF FIXED CYCLE
TIME ON TEST DATA

No Removed	Outliers	Median Error
Node Mean		0.097
Node Median		0.090
12 Outliers Removed		Median Error
Node Mean		0.084
Node Median		0.076

in the creation of the regression trees from minimized squared loss to minimized absolute loss. An algorithm for implementing the node median as a predictor has been discussed and developed in the literature [15] but was not implemented in software. Although the objective function in the trees was not changed, we could change the prediction at a leaf node of a tree to a median. This also improved predictions. Not only was the effect of the most extreme cycle times mitigated, but the effect of other lots that were extreme within a specific tree node was also eliminated.

Fig. 2 displays a tree diagram created after removing lots whose cycle time exceeds three standard deviations of the mean cycle time for all lots in the training data set. We note that the tree structure is very similar to that of Fig. 1. Differences arise in the values of the splitting rules and a few nodes are split on different variables. The final predictions found in the terminal nodes of the tree are also quite similar; however, these slight differences in splits and predicted values led to improved prediction performance.

Table IV demonstrates the improvement in prediction when outliers are removed. There is also a marked improvement when using the node median instead of the node mean for prediction.

The use of historical data assumes that the training data used to build the model adequately represents the lots which are to be predicted. Because semiconductor manufacturing occurs in a dynamic environment, changes are to be expected. An advantage of a decision tree model is that it can quickly and easily be

rebuilt from the current data from the process. In the large data set available for our application, data used to build the model (training data) can be separate from data used to evaluate the model (testing data). Consequently, the testing data provides a benchmark for performance for the current model. When future lots are predicted, prediction errors are expected to follow the same distribution as the testing data. A sustained departure between the actual and predicted cycle times larger in magnitude than expected from the training data is a signal that the process has changed from the conditions used to build the model. Consequently, a rebuild should be necessary. A quantitative decision rule based on the residuals (differences between actual and predicted cycle time) could be developed to comprise a real-time monitor of the model. This type of monitor is common for statistical process control, but the same methods can be applied with small change to monitor the adequacy of the cycle-time model. Our work has not yet evaluated this complementary analysis, but we expect it would be an effective addition to any model derived from historical data.

VI. CONCLUSION

Predictions for lots currently in production can be obtained from similar lots that have already completed production. Regression trees provide a flexible tool to define important variables and define similarity between lots. The ability of a regression tree to handle both categorical and continuous predictor variables provides an advantage in prediction over other methods, such as nearest neighbor predictors and neural networks. Furthermore, variables need not be scaled to common units. Predictions are improved by using an unsupervised algorithm, such as clustering, to form similar groups of lots and then apply the tree algorithm to the clusters. Here, scaling of variables is an issue, and we typically standardized variables to unit standard deviation.

We also addressed issues of robustness of the regression tree predictor. Use of the node median instead of the node mean provides improved prediction. For noisy data common to manufacturing applications, a robust predictor has many advantages for modeling typical behavior.

REFERENCES

- [1] W. Hopp and M. Spearman, *Factory Physics*. Chicago, IL: Irwin, 1996.
- [2] S.-H. Chung and H.-W. Huang, "Cycle time estimation for wafer fab with engineering lots," *IIE Trans.*, vol. 34, pp. 105–118, 2002.
- [3] J. C. Wood, "Cost and cycle time performance of fabs based on integrated single-wafer processing," *IEEE Trans. Semiconduct. Manufact.*, vol. 10, no. 1, pp. 98–111, Feb. 1997.
- [4] Y. D. Kim, J.-U. Kim, S.-K. Lim, and H.-B. Jun, "Due-date based scheduling and control policies in a multiproduct semiconductor wafer fabrication facility," *IEEE Tran. Semiconduct. Manufact.*, vol. 11, no. 1, pp. 155–164, 1998.
- [5] L. F. Atherton and R. W. Atherton, *Wafer Fabrication: Factory Performance and Analysis*. Boston, MA: Kluwer, 1995.
- [6] A. Raddon and B. Grigsby, "Throughput time forecasting model," in *Proc. IEEE/SEMI Advanced Semiconductor Manufacturing Conf.*, 1997, pp. 430–433.
- [7] S. T. Enns, "A dynamic forecasting model for Job shop flowing prediction and tardiness control," *Int. J. Production Res.*, vol. 33, no. 5, pp. 2817–2834, 1995.
- [8] M. Janakiram, P. Backus, S. Movafagh, and G. Runger, "Data mining for cycle time prediction," presented at the *IEEE Trans. CAS. II, INFORMS Annu. Conf.*, (in presented at) Atlanta, GA, 2003, unpublished.
- [9] G. Meng and S. Heragu, "Batch size modeling in a multi-item, discrete manufacturing system via an open queuing network," (in to appear) *IIE Trans.*.
- [10] A. C. Kaplan and A. T. Unal, "A probabilistic cost-based due date assignment model for job shops," *Int. J. Production Res.*, vol. 31, no. 12, pp. 2817–2834, 1993.
- [11] D.Y. Liao and C.N. Wang, "Neural-network-based delivery time estimates for prioritized 300-mm automatic material handling operations," *IEEE Trans. Semiconduct. Manufact.*, vol. 17, no. 3, pp. 324–332, 2004.
- [12] R. A. Johnson and D. W. Wichern, *Applied Multivariate Statistical Analysis*. Upper Saddle River, NJ: Prentice-Hall, 2002.
- [13] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. New York: Springer, 2001.
- [14] T. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.
- [15] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.



Phillip Backus received the M.S. degree in statistics from Arizona State University, Tempe.

He currently works in an operations research group for Nissan North America, Gardena, CA.



Mani Janakiram received the Ph.D. degree in industrial engineering from Arizona State University, Tempe.

He is a Process Control Manager at Intel Corporation, Chandler, AZ. He has more than 18 years of experience and has published more than 30 papers in the areas of statistical modeling, capacity modeling, data mining, factory operations research, and process control.



Shahin Mowzoon received the B.S. degree in electrical engineering from the University of Arizona, Tucson.

He is currently working at Intel Corporation, Chandler, AZ, in an operational research organization while pursuing his graduate engineering studies in multivariate analysis.



George C. Runger received degrees in industrial engineering and statistics.

He is a Professor in the Department of Industrial Engineering, Arizona State University, Tempe. His research interests include real-time monitoring and control, data mining, and other data-analysis methods with a focus on large, complex, multivariate data streams. In addition to academic work, he was a Senior Engineer at IBM.



Amit Bhargava received the B.S. degree in mechanical engineering from the Delhi College of Engineering, Delhi, India, and the M.S. degree in industrial engineering from the Arizona State University, Tempe.

He currently works as a Project Engineer for Honeywell International's Aerospace Repair and Overhaul Engineering group, Phoenix, AZ.